

Neural-driven Computing on a Minimal RISC-V ISA: Iterative ISA, Toolchain, and Runtime Co-Design

Bernhard Guillon

Advanced Systems Engineering (ASE2026)

Paris Lodron University of Salzburg

Salzburg, Austria

Email: Bernhard.Guillon@stud.plus.ac.at

Abstract—This paper reports an iterative implementation of a neural-centric execution path on a minimal RISC-V-like environment. The work spans an end-to-end toolchain from PyTorch model export to custom assembly, linking, execution on a C++ emulator and Verilator RTL model, and differential validation between both backends. We describe a phased ISA extension strategy (x77 and x7b custom neural instructions), a Rust-based assembler integrated as a drop-in backend, and a correctness-first verification methodology (unit tests, blackbox tests, assembler parity scaffolding against GNU tools, and emulator-vs-Verilator differential checks). The reported optimization series culminates in the x7b PMAC4 variant (nmatvec8xp4), reducing Verilator cycle threshold from 400k (8-lane baseline) to 150k while preserving deterministic framebuffer output parity.

Index Terms—custom ISA extension, RISC-V, Verilator, emulator, neural inference, differential validation

PROJECT CONTEXT

Supervisor: Univ.-Prof. Dipl.-Inform. Dr.-Ing. Christoph Kirsch

Course: ASE2026 (Advanced Systems Engineering)

University: Paris Lodron University of Salzburg

Date: 2026-03-29

A. Introduction

Modern AI deployments are commonly embedded in large software stacks. In contrast, this project investigates how far a compact execution stack can be pushed when neural operations are represented as first-class ISA extensions. The objective is not to compete with full ML frameworks, but to design and validate a minimal, inspectable path from trained model weights to executable program behavior on a custom compute substrate.

This paper summarizes the implemented system in this repository: model export, binary serialization, compiler and assembler integration, custom neural instructions, RTL realization, and verification. The narrative is grounded in measured and reproducible artifacts, including cycle-threshold benchmark JSON files and automated parity tests.

B. Scope and Exclusions

The implemented and evaluated workload is the neural character generation path (input character code to framebuffer

glyph output). The separate character recognition model exists in the repository but is intentionally out of scope for this paper because it was not part of the final optimization and validation campaign reported here.

C. System Overview

The system is a co-designed stack with six layers:

1. Training/export artifacts (PyTorch model checkpoint).
2. Weight-export pipeline to JSON and compact binary.
3. Model compiler that emits RISC-V assembly and neural custom mnemonics.
4. Rust assembler (`rv32as`) plus GNU linker flow to ELF binaries.
5. Runtime backends: C++ emulator and Verilator RTL runner.
6. Test and benchmark harnesses for parity and cycle threshold evaluation.

The operational data path is:

```
a0 character input -> one-hot mapping ->
layered matvec + activation -> clamp/u8 ->
framebuffer
```

D. Methodology: Iterative Co-Design

We used a strict iterative method:

1. Introduce one ISA/backend change (new mnemonic/opid/lane mode).
2. Extend compiler, assembler, C++ oracle mapping, RTL, and tests together.
3. Re-run targeted tests (encoding, execution, differential).
4. Re-run full regression gate.
5. Measure cycle-threshold impact and compare to previous phase.

This loop was repeated through x7b lane variants up to PMAC4. The key design principle was behavior stability with isolated performance changes: semantic output must remain identical while implementation throughput evolves.

E. Toolchain from PyTorch to Executable Neural Binary

1) *Stage A: Export and Intermediate Representation:* The weight-export pipeline (`projects/weight-export`) converts trained PyTorch models into:

- Intermediate JSON (human-readable, debuggable).
- Compact binary format (NRAL header, layer table, packed float32 weights/biases).

The format and conversion logic are implemented in `model_formats.py`, `export_generator.py`, and related loader notes. Weight transposition is performed to match efficient row-major inference access patterns.

2) *Stage B: Model Compiler:* `projects/emulator/model_compiler.py` loads exported JSON and emits assembly with `.incbin` model blobs and execution code. The interactive compiler path (`model_compiler_interactive.py`) drives framebuffer-oriented inference loops and supports neural opcode/lane mode selection:

- x77 custom opcode path.
- x7b enhanced path with lane modes: base, 4x, 8x, 8xpmac, 8xpmac2, 8xpmac3, 8xpmac4.

3) *Stage C: Assembly and Linking:* The repository uses a custom Rust assembler (`projects/rv32_assembler`) with GNU-compatible targets where possible, then links to ELF using GNU binutils. The CMake flow builds neural ELF variants such as:

- `neural-op-enhance.elf`
- `neural-op-enhance4.elf`
- `neural-op-enhance8.elf`
- `neural-op-enhance8xpmac*.elf`

F. Custom Assembler and GNU Parity Strategy

`rv32as` provides deterministic encoding for RV32I, RV32F, and the project neural custom instruction family. Parser/encoder support was extended for mnemonics including `nmatvec8xp4.f32` (opid 9 on x7b).

Parity methodology:

- Byte-level `.text` comparison against GNU assembler output via `objcopy`.
- Scripted parity checks (`projects/rv32_assembler/test_parity.sh`).
- Integrated parity fixture/testing in emulator CMake (`parity/... tests`).
- Additional neural instruction blackbox encoding tests (`test_neural_instr_assembler.py`).

This ensured encoding correctness while enabling project-specific instructions not available in stock GNU toolchains.

G. Neural ISA Extension Rationale and Evolution

The ISA extension progression used CUSTOM0 (0x77) and CUSTOM3 (0x7b):

- Core ops: matvec, ReLU, sigmoid piecewise-linear, clamp-to-u8.
- Enhanced x7b variants introduced lane and PMAC scheduling alternatives.

Variant	Threshold cycles
x7b-8lane	400,000
x7b-8lane-pmac	200,000
x7b-8lane-pmac2	200,000
x7b-8lane-pmac3	200,000
x7b-8lane-pmac4	150,000

The motivation was to reduce memory-access and accumulation overhead in the matvec hotspot without changing model semantics.

Notable progression:

- `nmatvec8x` for wider lane handling.
- `nmatvec8xp`, `nmatvec8xp2`, `nmatvec8xp3` for PMAC scheduling attempts.
- `nmatvec8xp4` with true PMAC4 path (including fourth memory read port in RTL).

H. Verification Methodology

Verification is multi-layered:

1. Unit tests for parser/encoder and runtime decode/execute.
2. Compiler/assembler blackbox tests for mnemonic emission and encoding.
3. Differential backend validation: `emulator_runner` vs `verilator_runner` output/return/framebuffer parity.
4. Full regression via CTest gate before accepting each phase.

Verilator differential validation is explicitly wired in CMake (`python/verilator/differential_validation`) and neural enhance variants are tested across configured lane modes.

I. Optimization Results

Cycle threshold results are extracted from repository benchmark artifacts (`docs/research/phase23/24/25-neural-lane-cycle-comp`)

1) *Verilator backend (phase progression summary):* Observed speedups at Phase 25:

- PMAC4 vs 8-lane: $400k / 150k = 2.67\times$
- PMAC4 vs PMAC/PMAC2/PMAC3: $200k / 150k = 1.33\times$

2) *C++ backend:* For the evaluated thresholds in this setup, x7b lane/PMAC variants remained flat at 1500-cycle threshold. This indicates the dominant gain in this phase was captured in RTL scheduling/bandwidth behavior rather than in the C++ oracle path.

J. Discussion

Three engineering outcomes were most relevant:

1. Co-evolution of compiler/assembler/RTL was necessary; isolated changes were insufficient.
2. Differential validation prevented backend drift while enabling aggressive micro-architectural iteration.
3. PMAC3 plateaued, while PMAC4 introduced a measurable gain due to additional memory bandwidth and scheduling structure.

K. Limitations and Threats to Validity

- Reported numbers are threshold-cycle based, not full wall-clock throughput.
- The workload is a compact FC pipeline; results may differ for larger models.
- Custom op semantics are project-specific and require toolchain coupling.
- Some parity paths (for broad assembler replacement) remain scaffolded for further hardening.

L. Conclusion

This project demonstrates that a lightweight neural-execution stack can be built and optimized through iterative ISA/runtime co-design while maintaining strict correctness gates. The final PMAC4 x7b variant delivered meaningful RTL speedup with preserved observable behavior. Equally important, the repository now contains a reproducible methodology: export pipeline, custom assembler, differential verification, and phase-based benchmarking.

M. Acronyms

- ABI: Application Binary Interface
- CI: Continuous Integration
- CTest: CMake test driver
- ELF: Executable and Linkable Format
- FC: Fully Connected (neural layer)
- ISA: Instruction Set Architecture
- MAC: Multiply-Accumulate
- PMAC: Parallel Multiply-Accumulate
- RTL: Register Transfer Level
- WIP: Work In Progress

N. References

- [1] RISC-V International, "The RISC-V Instruction Set Manual," <https://riscv.org/technical/specifications/>
- [2] Verilator project, "Verilator Open-Source SystemVerilog Simulator," <https://www.veripool.org/verilator/>
- [3] PyTorch contributors, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," NeurIPS, 2019.
- [4] GNU Binutils, "GNU assembler (as) and objcopy documentation," <https://sourceware.org/binutils/>
- [5] Repository artifact, `docs/research/phase25-neural-lane-cycle-comparison.json`.
- [6] Repository source, `projects/rv32_assembler/README.md` and `projects/rv32_assembler/src/{parser.rs,encoder.rs}`.
- [7] Repository source, `projects/emulator/model_compiler.py`, `projects/emulator/model_compiler_interactive.py`, `projects/emulator/blackbox_tests/test_verilator_differential.py`.